

FAILLES DE SÉCURITÉ ET 2FA

PROTOCOLES D'AUTHENTIFICATION

INTRODUCTION

Dans le développement d'une API ou d'une application web, l'authentification (savoir qui se connecte) n'est que la moitié du chemin.

Une fois le mécanisme de session ou de JWT en place, le canal de communication et le navigateur deviennent les cibles des attaques.

Sécuriser une application consiste à anticiper le fait que le client évolue dans un environnement potentiellement hostile.

Nous allons étudier les trois grandes failles architecturales du web et les parades modernes pour s'en prémunir.



CROSS-SITE SCRIPTING (XSS)

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

CONCEPT

Exploiter une faille XSS consiste à **injecter un script JS malveillant** dans un site web pour être, ensuite, exécuté par le navigateur de la victime.

Comme le JS s'exécute dans le contexte de la victime, le script pirate a accès à tout ce que l'utilisateur peut faire :

- Lire le [localStorage](#) ;
- Intercepter les saisies au clavier ;
- Envoyer des requêtes à l'API à l'insu de l'utilisateur ;
- Etc.

EXEMPLES DE VULNÉRABILITÉ

L'erreur classique est de faire aveuglément confiance aux entrées de l'utilisateur (ex: un commentaire, un nom de profil) et de l'afficher dans le HTML sans plus de vérification.

Côté backend

```
server.js

// Route vulnérable aux failles XSS
app.get('/search', (req, res) => {
  const query = req.query.q; // L'utilisateur saisit sa recherche
  // Problème : On injecte directement la chaîne brute dans le HTML
  res.send(`
    <h1>Résultats de recherche pour : ${query}</h1>
  `);
});
```

Côté frontend

```
script.js

// innerHTML interprète les balises HTML et les scripts
document.getElementById('zone-commentaire').innerHTML = data.text;
```

PARADES

Côté backend

Valider et échapper les caractères spéciaux (<, >, &, ", ') à l'aide de bibliothèques dédiées (ex: [express-validator](#) ou [DOMPurify](#)).

Côté frontend

Utiliser systématiquement `textContent` ou `innerText` qui neutralisent le code en le transformant en simple chaîne de caractères inoffensive.



CROSS-SITE REQUEST FORGERY (CSRF OU XSRF)

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

CONCEPT

Le CSRF (ou XSRF) est une attaque qui consiste à **forcer la victime à exécuter une action** sur un site web dans lequel elle est authentifiée.

Contrairement au XSS, le pirate ne peut pas lire les données du site.

Il profite, en réalité, d'une mécanique du navigateur web faisant qu'il joint **automatiquement** les cookies d'un site dès qu'il lui envoie une requête même si cette requête **part d'un site malveillant**.

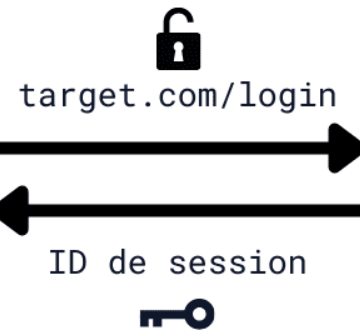
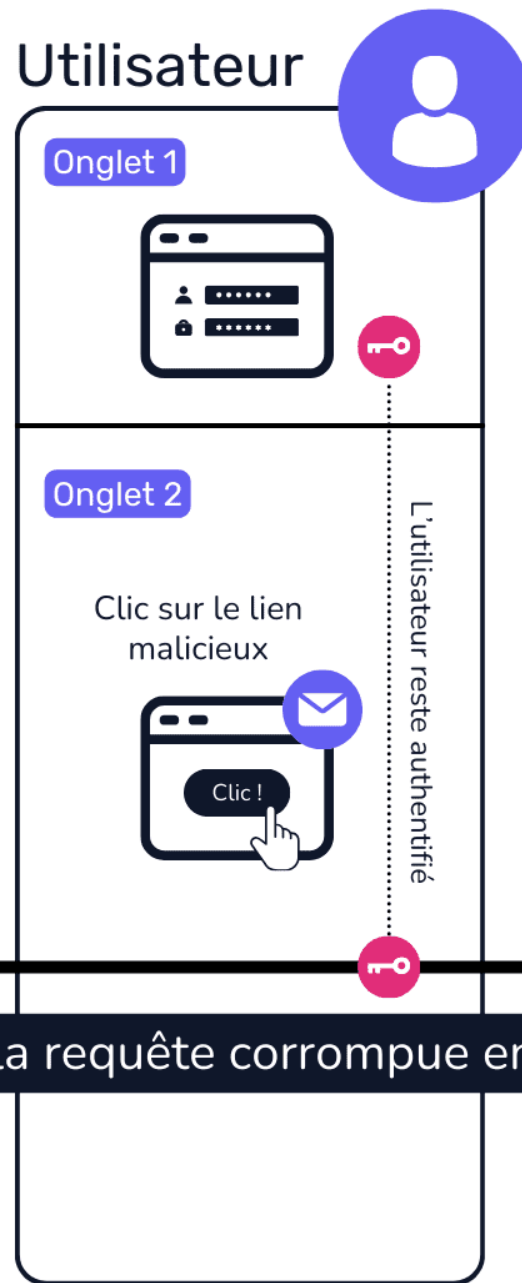
SCÉNARIO D'ATTAQUE

1. Vous êtes connecté à votre site bancaire qui vous authentifie avec un **cookie de session** ;
2. Dans un autre onglet, vous visitez un **site piégé** ;
3. Ce site contient un **formulaire invisible** qui s'envoie automatiquement vers le site bancaire.
4. Votre navigateur s'exécute, **associe votre cookie de session valide** et la banque valide l'**action** (ex. virement) à votre insu !

Hacker

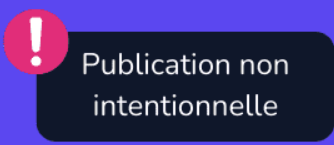


Utilisateur



Authentification

Session d'authentification créée



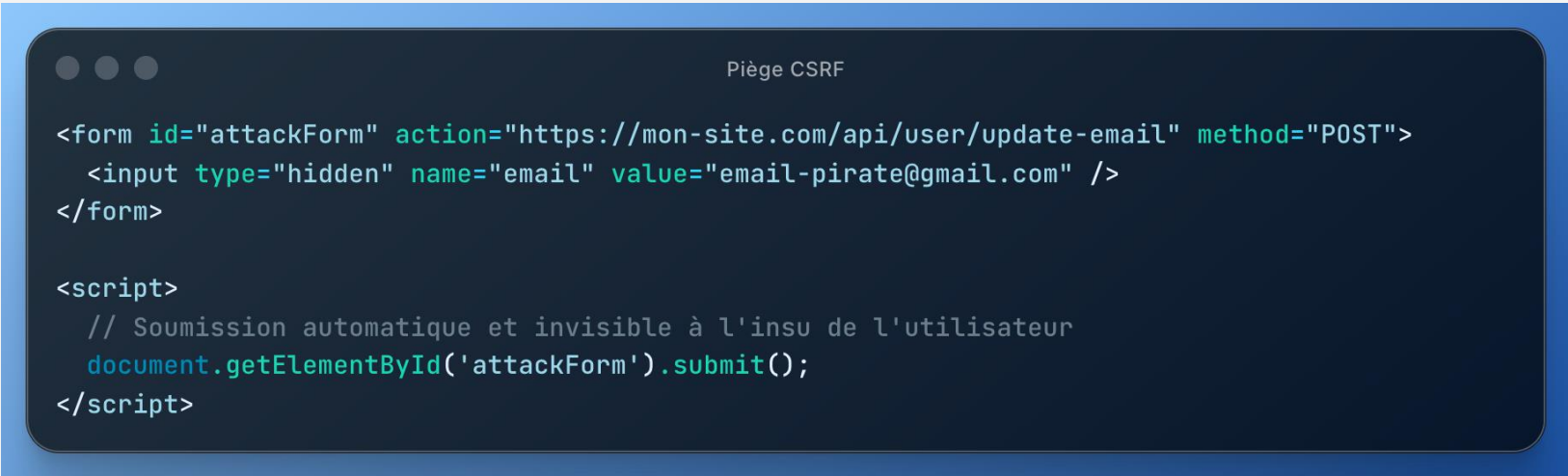
target.com



Exécution de la requête corrompue en arrière plan

LE PIÈGE CSRF

Le pirate crée une page web attractive (ex: "Vous avez gagné un iPhone !"). Dès que la victime connectée charge cette page piégée, un formulaire invisible est soumis en arrière-plan en JS :

A screenshot of a web browser window with a dark blue title bar and a dark background. The title bar has three small circles on the left and the text 'Piège CSRF' on the right. The main content area displays HTML and JavaScript code in a light blue monospace font. The code consists of a form with a hidden input field and a script that automatically submits the form.

```
<form id="attackForm" action="https://mon-site.com/api/user/update-email" method="POST">
  <input type="hidden" name="email" value="email-pirate@gmail.com" />
</form>

<script>
  // Soumission automatique et invisible à l'insu de l'utilisateur
  document.getElementById('attackForm').submit();
</script>
```

Le navigateur de la victime envoie la requête vers le serveur et **joint le cookie** (de session ou JWT) car il appartient au domaine `mon-site.com`. Le serveur valide la requête et l'e-mail du compte est modifié.

PARADES

Pour contrer ce mécanisme automatique, les architectures historiques utilisaient un jeton aléatoire unique, éphémère et lié à la session utilisateur : **le jeton CSRF**. Injecté dans un formulaire HTML ou dans les en-têtes de l'application, il permettait de valider toute action sensible.

Ainsi, lors d'une action (POST, PUT), le serveur exige **à la fois** le cookie ET le jeton CSRF.

Comme le site pirate est incapable de deviner ou de lire ce jeton unique, l'attaque échoue.

Aujourd'hui, On préfère utiliser la directive `SameSite: 'strict'` qui règle le problème directement au niveau du protocole réseau du navigateur, sans alourdir le code :

```
Parade CSRF

res.cookie('token', token, {
  httpOnly: true,
  secure: true,
  sameSite: 'strict' // 🖱️ LA PARADE
})
```



MAN-IN-THE-MIDDLE (MITM)

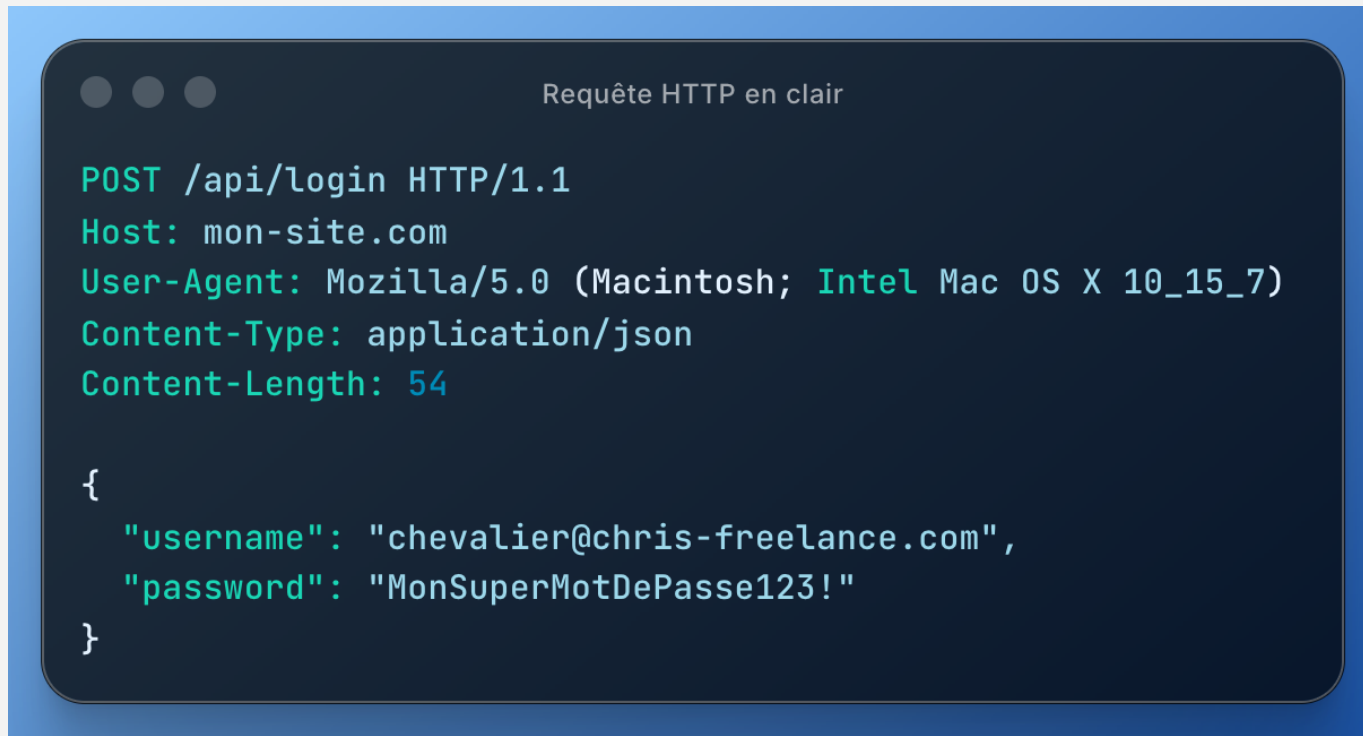
CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

CONCEPT

L'attaque « Man in the Middle » (ou *de l'homme au milieu*) se produit lorsqu'un attaquant s'interpose sur le réseau entre le client et le serveur.

Cette situation est fréquente sur les réseaux Wi-Fi publics non sécurisés (cafés, gares). Si les données **transitent en texte clair**, le pirate peut écouter le trafic, intercepter les identifiants au moment du login, ou capturer le JWT qui transite, par exemple.



```
Requête HTTP en clair

POST /api/login HTTP/1.1
Host: mon-site.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
Content-Type: application/json
Content-Length: 54

{
  "username": "chevalier@chris-freelance.com",
  "password": "MonSuperMotDePasse123!"
}
```

PARADES

Il n'existe aucune parade logicielle au niveau du code pour empêcher l'interception sur un réseau. La seule défense est le **chiffrement du protocole de transport**.

L'utilisation obligatoire de **HTTPS** (via des certificats SSL/TLS, comme ceux fournis gratuitement par [Let's Encrypt](#)) garantit que toutes les données (URL, en-têtes, corps de la requête, cookies) sont **chiffrées** avant de quitter le client.

Même si un pirate intercepte le paquet au milieu, il ne verra qu'une suite de caractères mathématiquement indéchiffrable (sans la clé secrète).

HTTP VS HTTPS

```
[14:29:35.937][127.0.0.1:51033] client connect
[14:29:35.988][127.0.0.1:51033] server connect example.com:80 (172.66.147.243:80)
127.0.0.1:51033: GET http://example.com/
<< 200 OK 559b
[14:29:36.032][127.0.0.1:51033] client disconnect
[14:29:36.032][127.0.0.1:51033] server disconnect example.com:80 (172.66.147.243:80)
[14:30:02.053][127.0.0.1:51035] client connect
[14:30:02.294][127.0.0.1:51035] server connect example.com:443 (172.66.147.243:443)
```

```
=====  
[MITM] PAQUET BRUT INTERCEPTÉ (DEPUIS LE CLIENT) [MITM]  
Taille du paquet : 101 octets  
=====
```

APERÇU EN HEXADÉCIMAL :

```
50 52 49 20 2a 20 48 54 54 50 2f 32 2e 30 0d 0a  
0d 0a 53 4d 0d 0a 0d 0a 00 00 12 04 00 00 00 00  
00 00 03 00 00 00 64 00 04 02 00 00 00 02 00  
00 00 00 00 00 04 08 00 00 00 00 00 01 ff 00 01  
00 00 1c 01 05 00 00 00 01 82 84 87 41 88 2f 91  
d3 5d 05 5c 87 a7 7a 88 25 b6 50 c3 ab bc ea e0  
53 03 2a 2f 2a
```

TENTATIVE DE LECTURE EN TEXTE BRUT (ASCII) :

```
PRI * HTTP/2.0....SM.....d.....A./..].\...z.%P.....S.*/*...
```

```
=====  
127.0.0.1:51035 -> tcp -> example.com:443  
127.0.0.1:51035 <- tcp <- example.com:443  
=====
```

```
=====  
[MITM] PAQUET BRUT INTERCEPTÉ (DEPUIS LE CLIENT) [MITM]  
Taille du paquet : 9 octets  
=====
```

APERÇU EN HEXADÉCIMAL :

```
00 00 00 04 01 00 00 00 00
```

TENTATIVE DE LECTURE EN TEXTE BRUT (ASCII) :

```
.....  
=====
```

```
127.0.0.1:51035 -> tcp -> example.com:443  
127.0.0.1:51035 <- tcp <- example.com:443  
127.0.0.1:51035 <- tcp <- example.com:443  
[14:30:03.368][127.0.0.1:51035] server disconnect example.com:443 (172.66.147.243:443)  
[14:30:03.369][127.0.0.1:51035] client disconnect
```


EXPLICATIONS

En haut : La vulnérabilité du HTTP (texte en clair)

On observe `GET http://example.com/ << 200 OK 559b`

👉 La requête transite mais l'outil [mitmproxy](#) intercepte et lit l'action en texte brut.

⚠️ Si un utilisateur transmet un cookie sur ce canal, l'attaquant n'a qu'à lire l'écran pour voler les données (dans le *body*).

En bas : Le bouclier HTTPS / TLS (flux chiffré)

On observe une nouvelle connexion.

👉 Le texte lisible disparaît au profit d'une suite d'octets.

Hexadécimal : `00 00 00 04 01 00 00 00 00`

ASCII : `.....`

🔒 Le tunnel TLS (pour *Transport Layer Security*) *1 est actif. Les données de l'application (en-têtes HTTP, cookies, corps du message) sont chiffrées donc illisibles pour l'intercepteur.



STRATÉGIES DE SÉCURISATION

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

LE RENFORCEMENT DES COOKIES

Précédemment, nous avons fait le choix de stocker nos JWT dans des cookies plutôt que dans le localStorage pour être protégé des **failles XSS**. Cependant, un cookie doit être configuré avec une extrême précision via trois directives de sécurité :

```
server.js

res.cookie('token', token, {
  httpOnly: true,
  secure: true,
  sameSite: 'strict'
});
```

RAPPEL DES RÔLES

1. `httpOnly: true` (**parade XSS**) : Interdit à tout code JS d'accéder au cookie via la commande `document.cookie`. Ainsi, si un attaquant réussit une injection XSS, il ne pourra pas voler la clé de session ou le JWT.
2. `secure: true` (**parade MITM**) : Ordonne au navigateur de n'envoyer ce cookie que si la requête est explicitement en HTTPS. Le cookie ne transitera jamais en clair sur le réseau.
3. `sameSite: 'strict'` (**parade moderne Anti-CSRF**) : Indique au navigateur que le cookie ne doit *jamais* être envoyé si la requête provient d'un site tiers. Cela neutralise le vecteur d'attaque CSRF sans nécessiter l'implémentation d'un jeton CSRF.

SECURITY HEADERS

Le serveur Express peut (et doit) envoyer des instructions au navigateur pour lui imposer des règles de comportement strictes. C'est ce qu'on appelle les **Security Headers**. Le site de référence [Security Headers](#) permet de scanner une API et de lui attribuer une note de A à F.

La politique la plus importante à implémenter, [CSP](#) (**Content Security Policy**), permet d'indiquer au navigateur une liste des domaines à partir desquels il a le droit de charger des fichiers. Si un script XSS tente de s'exécuter depuis un domaine pirate, le navigateur bloque la requête d'origine.

En Node.js, l'intégration du middleware [helmet](#) configure automatiquement ces en-têtes aux normes de l'industrie :

```
server.js  
  
const helmet = require('helmet');  
app.use(helmet());
```

[Code à copier](#)

SANS HELMET VS AVEC HELMET

✖ Requête sans Helmet

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 54
```

- `X-Powered-By: Express` indique la technologie utilisée. Si une faille sort sur Express, le pirate sait immédiatement qu'il peut cibler ce site.
- De plus, aucune directive de protection n'est donnée au navigateur.

✔ Requête avec Helmet

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 54
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 0
Content-Security-Policy: default-src 'self';...
Strict-Transport-Security: max-age=15552000; includeSubDomains
```

- Helmet supprime la signature Express ;
- `X-Content-Type-Options: nosniff` force le navigateur à respecter le [MIME type](#) envoyé par le serveur ;
- `X-Frame-Options: SAMEORIGIN` interdit d'intégrer l'application dans une [iframe](#) tierce (parade contre le [Clickjacking](#)).
- `Content-Security-Policy (CSP)` contient la liste blanche qui dicte au navigateur d'où les scripts ont le droit de provenir et d'être exécutés.

PROTECTION CONTRE LES ATTAQUES PAR REJEU (OU *REPLAY ATTACKS*)

Une attaque *replay attack* consiste, pour un pirate, à intercepter une requête valide et à la renvoyer plusieurs fois d'affilée pour saturer ou frauder le serveur.

Pour s'en prémunir, on met en place des **short-lived JWT** (ou *jeton à vie courte*).

Idéalement, **un jeton d'accès ne doit durer que quelques minutes** (entre 15 et 60). Ainsi, si un token est intercepté, sa fenêtre d'utilisation par un pirate sera réduite.

QUESTION OUVERTE

Si j'intercepte une requête HTTPS et que je vois une suite d'octets illisibles, le mot de passe que l'utilisateur a tapé a-t-il été haché par le navigateur avant d'être chiffré ?

Répondez dans le chat !

A decorative wavy line in light blue and white on the left side of the slide.

AUTHENTICATION MULTI-FACTEURS (MFA)

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

LE MOT DE PASSE EST MORT

L'authentification traditionnelle repose sur un facteur unique : **ce que l'utilisateur sait** (son mot de passe). Aujourd'hui, ce modèle est considéré comme insuffisant face aux attaques modernes :

- **L'hameçonnage** (ou *phishing*) : L'utilisateur donne lui-même ses identifiants depuis une fausse interface ;
- **Le credential stuffing** : Les pirates testent en masse des milliards d'identifiants fuités sur d'autres sites, profitant du fait que les utilisateurs réutilisent souvent le même mot de passe ;
- **Les fuites de bases de données** : Même hachés avec du sel, les mots de passe finissent par être attaqués.

Pour pallier cela, l'industrie a généralisé l'**Authentification Multi-Facteurs (MFA)** ou **Double Facteurs (2FA)**.

LES 3 PILIERS DE L'AUTHENTIFICATION

Pour qu'une authentification soit qualifiée de "Multi-Facteurs", elle doit combiner au moins deux éléments parmi les trois piliers historiques de la sécurité :

1. **Le facteur de connaissance** (ce que je sais) : Un mot de passe, un code PIN ou la réponse à une question secrète ;
2. **Le facteur de possession** (ce que j'ai) : Un smartphone, une clé de sécurité USB ([Yubikey](#)) ou une carte SIM (réception de SMS) ;
3. **Le facteur d'inhérence** (ce que je suis) : Une empreinte digitale, la reconnaissance faciale ou le scan rétinien.

 Recevoir un mot de passe par e-mail ET utiliser son mot de passe habituel **n'est pas** une MFA. L'e-mail et le mot de passe web relèvent tous deux du **facteur de connaissance**.

L'ALGORITHME TIME-BASED ONE-TIME PASSWORD (OU TOTP)

- Le [TOTP](#) (ou *mot de passe unique basé sur le temps*) est la méthode **2FA** la plus répandue. Elle présente l'avantage d'être **stateless** et de fonctionner **hors-ligne**.

L'algorithme TOTP (défini par la [RFC 6238](#)) est une déclinaison de l'algorithme HOTP (pour *HMAC-based One-time Password* ; basé sur un compteur). Au lieu d'un compteur de clics, TOTP utilise le **temps universel** ([Timestamp Unix](#)) : valeur en seconde depuis le 1^{er} janvier 1970.

La recette repose sur deux ingrédients uniques partagés secrètement entre le serveur et l'application mobile d'authentification (ex. [Microsoft Authenticator](#) ou [Google Authenticator](#)) lors de l'affichage du QR code (pour *Quick Response code*) initial :

- La clé secrète** (K) : Une suite d'octets générée aléatoirement par le serveur, stockée en BDD pour l'utilisateur, et transmise au smartphone via le QR code (encodée en [Base32](#)) ;
- Le compteur de temps** (C) : Un nombre entier calculé à partir du timestamp actuel. On prend le temps universel en secondes (T) et on le divise par une fenêtre de temps (30 secondes), arrondi à l'inférieur :

$$C = \left\lfloor \frac{T}{30} \right\rfloor$$

LA GÉNÉRATION DU CODE À 6 CHIFFRES

Toutes les 30 secondes, l'application mobile d'authentification et le serveur exécutent exactement la même recette mathématique :

1. Calcul du temps

On ne veut pas que le code change à chaque milliseconde. L'algorithme prend donc le temps universel et le divise par l'intervalle choisi (souvent 30 secondes) et l'arrondit à l'inférieur.

👉 S'il est 14h00m15s, puis 14h00m28s, le résultat de la division restera le même. Le code ne change pas. Dès qu'on passe à 14h00m30s, le résultat change et le code aussi.

2. Le mixage

On mélange la clé secrète et ce bloc de temps en utilisant une fonction de hachage (souvent [SHA-1](#)). Ce mélange s'appelle un [HMAC](#).

3. La réduction (*dynamic tuncation*)

Le résultat du mixage est une longue suite de caractères (40 caractères hexadécimaux) impossible à saisir en moins de 30 secondes pour un humain. L'algorithme extrait **4 octets** de cette suite, les transforme en un nombre entier et applique un modulo **10** pour ne garder que les **6 derniers chiffres**.

UNE SÉCURITÉ MAXIMALE

Aucun échange réseau

👉 Le code change toutes les 30 secondes sans qu'aucune donnée ne circule entre le téléphone et le serveur. Ils font juste le même calcul mathématique chacun de leur côté au même moment.

La tolérance réseau

👉 Si l'heure du serveur et celle du smartphone sont légèrement décalées (de quelques secondes), le validateur du serveur teste généralement le compteur actuel (C), mais aussi le précédent ($C - 1$) et le suivant ($C + 1$) pour éviter de bloquer l'utilisateur.

LA MFA EST OBLIGATOIRE ?

Non, la MFA n'est pas légalement obligatoire pour tous les sites mais [recommandée par la CNIL](#).

Les secteurs où elle est actuellement obligatoire :

- **Le secteur bancaire et financier** (en Europe) : La [DSP2](#) (**Directive révisée sur les Services de Paiement**) impose l'[Authentification Forte du Client](#) (SCA). C'est pourquoi vous devez obligatoirement valider vos accès bancaires ou vos achats en ligne via l'application de votre banque (combinaison de la possession du téléphone et de l'inhérence via l'empreinte faciale/digitale) ;
- **Les infrastructures critiques et [Opérateurs d'Importance Vitale](#) (OIV)** : Les administrations, hôpitaux ou industries de l'énergie ont l'obligation d'activer la MFA pour tous les accès distants de leurs employés ;
- **La santé** : Les hébergeurs de données de santé imposent des barrières d'accès MFA pour les professionnels de santé.

ET LE RESTE DU WEB ?

Pour un site e-commerce standard, un réseau social ou un SaaS de productivité, la MFA n'est pas une obligation légale pour des raisons d'**UX**.

Imposer la MFA à un utilisateur qui veut simplement acheter une paire de chaussures ou poster une photo crée une "friction". Si le processus est trop lourd, l'utilisateur abandonne le site. La bonne pratique consiste donc à :

- **Proposer la MFA en option** pour offrir la possibilité aux utilisateurs soucieux de leur sécurité de l'activer ;
- **L'authentification adaptative** pour que le site ne demande pas de deuxième facteur en temps normal mais **déclenche une alerte MFA** si une anomalie est détectée (ex: connexion depuis un nouvel appareil, un nouveau pays ou après plusieurs tentatives de mots de passe ratées).

QUESTION OUVERTE

Si un pirate réussit à voler le mot de passe d'un utilisateur en clair mais que cet utilisateur a activé la 2FA, quel code de statut va renvoyer la route de connexion au pirate ?

Répondez dans le chat !



IMPLÉMENTATION

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

ENVIRONNEMENT DE DÉVELOPPEMENT

Dans le cadre de cette démo, j'ai déjà initialisé plusieurs fichiers n'incluant aucune notion inédite :

- `config/db.js`
- `.env`
- `middlewares/checkAuth.js`
- `server.js`
- `views/dashboard.html`
- `public/dashboard.js`

👉 *Parcourons-les ensemble !*

Voici les dépendances à installer :

👉 `npm install express cookie-parser jsonwebtoken bcrypt better-sqlite3 dotenv qrcode @otplib/preset-v11`



BACKEND

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

ROUTER

```
routes/authRoutes.js

const express = require('express')
const router = express.Router()
const jwt = require('jsonwebtoken')
const path = require('path')
const authController = require('../controllers/authController')
const checkAuth = require('../middlewares/checkAuth')

// Routes d'authentification
router.post('/setup-2fa', authController.setup2FA)
router.post('/confirm-2fa', authController.confirm2FA)
router.post('/login', authController.login)
router.post('/verify-2fa', authController.verify2FA)
router.post('/logout', authController.logout)

// Routes protégées
router.get('/dashboard', checkAuth, (req, res) => {
  res.sendFile(path.join(__dirname, '../views', 'dashboard.html'))
})
router.get('/user/me', checkAuth, (req, res) => {
  res.json({ username: req.user.username })
})

router.get(/.*$/ , (req, res) => {
  res.redirect('/')
})

module.exports = router
```

[Copier le code](#)

ÉTAPES DE LA 2FA

1. Génération du QR Code (`POST /setup-2fa`) — *L' enrôlement (étape A)*

Initialise la 2FA pour un utilisateur connecté en créant une clé secrète unique (K) via otplib, l'enregistre de manière temporaire en BDD et la renvoie au frontend sous forme de QR code à afficher dans une `img`.

2. Confirmation de l'activation (`POST /confirm-2fa`) — *L' enrôlement (étape B)*

Finalise et verrouille le processus d'activation. L'utilisateur saisit le tout code affiché sur son téléphone. Si otplib le valide, le serveur passe définitivement une colonne `two_factor_enabled` à `1` en base.

3. Connexion (`POST /login`) — *Le premier verrou*

Vérifie le couple nom d'utilisateur (ou email)/mot de passe. Si le mot de passe est correct mais que la 2FA est obligatoire, la route intercepte la session, bloque la distribution du jeton d'accès et renvoie `{ requires2FA: true }` dans une réponse JSON.

4. Vérification du code TOTP (`POST /verify-2fa`) — *Le second verrou*

Valide le code à 6 chiffres généré par l'application mobile de l'utilisateur (Google Authenticator ou autre). Si le code calculé par otplib est correct, le serveur délivre enfin le cookie sécurisé contenant le jeton d'accès.

1 – GÉNÉRATION DU QR CODE

```
controllers/authController.js

const db = require('../config/db')
const bcrypt = require('bcrypt')
const jwt = require('jsonwebtoken')
const { authenticator } = require('@otplib/preset-v11')
const qrcode = require('qrcode')

exports.setup2FA = async (req, res) => {
  const { username } = req.body
  const secret = authenticator.generateSecret()

  const otpauth = authenticator.keyuri(username, 'Secure!', secret)

  db.prepare('UPDATE users SET two_factor_secret = ? WHERE username = ?').run(
    secret,
    username
  )

  const qrCodeImage = await qrcode.toDataURL(otpauth)

  res.json({ qrCode: qrCodeImage, secret: secret })
}
```

[Code à copier](#)

EXPLICATIONS

1. Génération du secret unique

👉 On génère une clé cryptographique unique (K). C'est la racine mathématique qui permettra de calculer les futurs codes à 6 chiffres.

2. Création de l'URI standardisée (`otpauth://`)

👉 On construit une chaîne de caractères ([URI](#)) normalisée que toutes les applications mobiles d'authentification savent décoder. Elle contient le nom de l'application (`Secure!`), l'identifiant de l'utilisateur et la clé secrète.

3. Sauvegarde temporaire en base

👉 On sauvegarde la clé secrète pour cet utilisateur. À ce stade, la colonne `two_factor_enabled` reste à `0`. Le système est en attente de confirmation pour ne pas verrouiller le compte par erreur.

4. Transformation en QR code visuel

👉 On convertit l'URI textuelle en une image encodée en Base32 (chaîne représentant l'image). Cela permet au frontend de l'afficher directement dans une balise HTML `img` pour que l'utilisateur puisse la scanner.

2 – CONFIRMATION DE L'ACTIVATION

```
controllers/authController.js

exports.confirm2FA = (req, res) => {
  const { username, code } = req.body
  const user = db
    .prepare('SELECT * FROM users WHERE username = ?')
    .get(username)

  const isValid = authenticator.check(code, user.two_factor_secret)

  if (!isValid) {
    return res
      .status(401)
      .json({ error: 'Code incorrect. Activation avortée.' })
  }

  db.prepare('UPDATE users SET two_factor_enabled = 1 WHERE username = ?').run(
    username
  )
  res.json({
    success: true,
    message: 'La 2FA est désormais activée sur votre compte !'
  })
}
```

[Code à copier](#)

EXPLICATIONS

Avec la même méthode `authenticator.check()`, le serveur extrait le `two_factor_secret` qui a été mis en attente à l'étape précédente et le confronte au tout premier code généré sur le smartphone de l'utilisateur.

On prouve ainsi que le téléphone et le serveur sont synchronisés **avant** de verrouiller le compte. Si l'heure du téléphone est décalée, le code échoue et l'activation 2FA est avortée.

Sinon, la colonne `two_factor_enabled` passe à **1**.

3 - CONNEXION

```
controllers/authController.js

exports.login = async (req, res) => {
  try {
    const { username, password } = req.body
    const user = db
      .prepare('SELECT * FROM users WHERE username = ?')
      .get(username)
    if (!user || !(await bcrypt.compare(password, user.password_hash))) {
      return res.status(401).json({ error: 'Identifiants incorrects' })
    }

    if (user.two_factor_enabled === 0 || !user.two_factor_secret) {
      return res.status(403).json({
        error: 'Accès refusé. La double authentification est obligatoire.'
      })
    }

    if (user.two_factor_enabled === 1) {
      return res.json({
        requires2FA: true,
        message: 'Étape 1 validée. Veuillez fournir votre code à 6 chiffres.',
        username: user.username
      })
    }
  } catch (error) {
    console.error('Erreur lors du login :', error)
    return res.status(500).json({ error: 'Erreur interne du serveur' })
  }
}
```

[Code à copier](#)

EXPLICATIONS

On importe les librairies permettant d'avoir accès au moteur cryptographique qui gère l'algorithme TOTP (génération de secrets, calculs temporels et vérifications de codes) et à la transformation de nos secrets d'authentification en QR code.

Le mot de passe est valide, mais le niveau de sécurité du compte est insuffisant !

On n'utilise pas le code de statut `401 - Unauthorized` mais `403 - Forbidden` pour marquer la rupture de sécurité. La fonction s'arrête sans distribuer de jeton d'accès ni de cookie.

Le drapeau `requires2FA: true` sert de signal au client pour basculer dynamiquement l'affichage vers l'étape 2 (le champ du code TOTP à 6 chiffres).

4 – VÉRIFICATION DU CODE TOTP

```
routes/authRoutes.js

// ...
exports.verify2FA = (req, res) => {
  const { username, code } = req.body
  const user = db
    .prepare('SELECT * FROM users WHERE username = ?')
    .get(username)
  if (!user) return res.status(404).json({ error: 'Utilisateur introuvable' })

  const isValid = authenticator.check(code, user.two_factor_secret)
  if (!isValid) {
    return res.status(401).json({ error: 'Code 2FA invalide ou expiré' })
  }

  const token = jwt.sign(
    { id: user.id, username: user.username },
    process.env.JWT_SECRET,
    {
      expiresIn: '15m'
    }
  )
  res.cookie('token', token, { httpOnly: true, sameSite: 'strict' })
  res.json({
    success: true,
    message: 'Authentification double facteur réussie !'
  })
}
// ...
```

[Code à copier](#)

EXPLICATIONS

La fonction magique `authenticator.check()` réexécute localement l'algorithme TOTP :

- Elle prend la clé secrète de l'utilisateur (`user.two_factor_secret`) stockée en base de données et la combine avec le **timestamp actuel du serveur** (arrondi à la tranche de 30 secondes la plus proche) ;
- Elle compare le résultat obtenu avec le code à 6 chiffres envoyé par l'utilisateur. Si les deux concordent, `isValid` devient `true`.

Si l'utilisateur saisit un code généré il y a plus de 30 secondes, le calcul basé sur le temps actuel du serveur ne correspondra plus (`401 Unauthorized`).



Ce n'est qu'une fois la barrière de `authenticator.check()` franchie avec succès que le serveur génère enfin le jeton d'accès et le dépose dans le cookie.



FRONTEND

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

0 – FORMULAIRE DE CONNEXION

```
public/index.html

<!DOCTYPE html>
<html lang="fr-FR">
  <head>
    <meta charset="UTF-8" />
    <title>Authentification à double facteurs</title>
  </head>
  <body>
    <h2>1. Connexion</h2>
    <label for="user">Nom d'utilisateur</label>
    <input type="text" id="user" placeholder="Nom d'utilisateur" />
    <label for="pass">Mot de passe</label>
    <input type="password" id="pass" placeholder="Mot de passe" />
    <button onclick="login()" class="btn btn-primary mt-2">Se connecter</button>
    <div
      id="zone2FA"
      style="display: none; margin-top: 20px; background: #f9f9f9; padding: 15px;"
    >
      <h3>🔑 Entrez votre code à 6 chiffres</h3>
      <label for="code2FA">Code à 6 chiffres</label>
      <input type="text" id="code2FA" placeholder="000000" />
      <button onclick="valid2FAConnection2FA()" class="btn btn-primary mt-2">
        Vérifier le code
      </button>
    </div>
    <hr style="margin: 30px 0" />
    <h2>2. Activer la 2FA</h2>
    <button onclick="chargeQR()" class="btn btn-primary">Générer le QR Code</button>
    <div id="qrZone" class="mt-3"></div>
    <script src="script.js"></script>
  </body>
</html>
```

[Code à copier](#)

1 – GÉNÉRATION DU QR CODE

```
public/script.js

let currentUsername = ''

async function chargeQR () {
  currentUsername = document.getElementById('user').value
  const res = await fetch('/setup-2fa', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username: currentUsername })
  })
  const data = await res.json()

  if (data.qrCode) {
    document.getElementById('qrZone').innerHTML = `
      <p>Scannez ce QR Code avec votre application d'authentification :</p>
      <br><br>
      <input type="text" id="confirmCode" placeholder="Code à 6 chiffres">
      <button onclick="confirm2FAActivation()">Confirmer l'activation</button>
    `
  }
}
```

[Code à copier](#)

2 – CONFIRMATION DE L'ACTIVATION

```
public/script.js

async function confirm2FAActivation () {
  const res = await fetch('/confirm-2fa', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      username: currentUsername,
      code: document.getElementById('confirmCode').value
    })
  })
  const data = await res.json()
  alert(data.message || data.error)
}
```

[Code à copier](#)

3 – CONNEXION

```
public/script.js

async function login () {
  currentUsername = document.getElementById('user').value
  const res = await fetch('/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      username: currentUsername,
      password: document.getElementById('pass').value
    })
  })
  const data = await res.json()

  if (data.requires2FA) {
    document.getElementById('zone2FA').style.display = 'block'
  } else if (data.success) {
    window.location.href = '/dashboard'
  } else {
    alert(data.error)
  }
}
```

[Code à copier](#)

4 – VÉRIFICATION DU CODE TOTP

```
public/script.js

async function valid2FAConnection2FA () {
  const res = await fetch('/verify-2fa', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      username: currentUsername,
      code: document.getElementById('code2FA').value
    })
  })
  const data = await res.json()

  if (data.success) {
    window.location.href = '/dashboard'
  } else {
    alert(data.error)
  }
}
```

[Code à copier](#)

BILAN

Voyons le **protocole 2FA (TOTP)** en **4 étapes** :

1. **L'enrôlement** (`POST /setup-2fa`) : Le serveur génère une clé secrète unique pour l'utilisateur connectée (`generateSecret()`). Elle est stockée en base et transmise au client sous forme de **QR code (URI `otpauth://` encodée en Base32)** ;
2. **La confirmation** (`POST /confirm-2fa`) : L'utilisateur scanne le code et saisit le premier code à 6 chiffres. Le serveur valide la synchronisation temporelle (`authenticator.check()`) et **verrouille le statut** en base `two_factor_enabled = 1` ;
3. **Le premier verrou** (`POST /login`) : Lors de la connexion, le serveur valide le mot de passe. Si la 2FA est activé, il **intercepte la session, bloque l'accès** et renvoie un drapeau `{ requires2FA: true }` au frontend ;
4. **Le second verrou** (`POST /verify-2fa`) : Le frontend affiche le champ intermédiaire. L'utilisateur saisit son code TOTP actuel. Si la validation cryptographique réussit, le serveur délivre le **cookie sécurisé contenant le jeton d'accès**.

QUIZ ORAL

1. Un site web vous demande votre mot de passe, puis vous envoie un lien de validation par e-mail sur lequel cliquer. Est-ce un deuxième facteur (MFA) au sens strict de la sécurité ?
2. Dans notre implémentation avec otplib, que contient concrètement le QR code affiché à l'écran lors de la route setup2FA ?
3. Si le smartphone de l'utilisateur est configuré en mode avion, l'application Google Authenticator peut-elle quand même générer un code à 6 chiffres valide pour notre API ?
4. Dans notre contrôleur login durci, si le mot de passe de l'utilisateur est correct mais qu'il n'a pas encore configuré sa 2FA, que renvoie immédiatement notre serveur ?



COMPLÉMENTS

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

*1 - TLS

Le TLS (pour *Transport Layer Security*) est un protocole de sécurisation des échanges interposé entre la couche **Transport (TCP)** et la couche **Applicative (HTTP)**. C'est lui qui transforme le HTTP (vulnérable) en HTTPS (sécurisé) grâce à 3 piliers de sécurité :

1. Le chiffrement

Il protège les données contre l'écoute indiscrete (Eavesdropping). Même si un pirate intercepte les paquets (comme on l'a vu sur mitmproxy), les données restent totalement illisibles.

2. L'authentification

Il garantit via des **certificats X.509** (délivrés par des autorités de certification comme [Let's Encrypt](#)) que le client communique bien avec le vrai serveur et non avec un imposteur.

3. L'intégrité

Il assure que les données n'ont pas été modifiées ou corrompues pendant leur voyage sur le réseau (grâce à des codes de vérification MAC/HMAC).



DES QUESTIONS ?

CHEVALIER@CHRIS-FREELANCE.COM